Information Theory and Entropy Coding

- Information Theory
- Huffman Coding
- Arithmetic Coding
- Probability Model

Information Theory Preliminary

- **What is a "Bit"?**
  - **Definition:** "Bit" is a contraction of binary digit, representing either a **0** or a **1**.
  - **Encoding Capacity**: 3 bits can represent 8 unique configurations ($2^3 = 8$)
    - Therefore, we can use 3 bits to encode any integer from 1 to 8.
  - **Logarithms and Information:**
    - In information theory, we use **base-2 logarithms** ($\log_2$)
    - $\log_2 8 = 3$ bits
    - $\log_2 1000 \approx 10$ bits (since $2^{10} \approx 1024$)
    - $\log_2 1000000 \approx 20$ bits (since $2^{20} \approx 1000000$)
- **If some symbols occur much more often than others, should every symbol use the same number of bits?**

Efficient Encodings: The Coin Flip Case

- **Does Probability Affect Bit Count?**
  - **Fair Coin:** Heads and Tails occur with equal probability → **P(H) = P(T) = 1/2**
    - All 2n sequences are equally likely
    - Therefore, identifying one sequence of n flips reqa million flips = 1,000,000 bits.
  - **Biased Coin:** Suppose **P(H) = 1/4, P(T) = 3/4**
    - In one million flips, a typical sequence contains approximately:
      - 250,000 Heads, 750,000 Tails
    - The number of such arrangements is approximately: $\binom{1{,}000{,}000}{250{,}000}$

    → Therefore, Identifying one sequence requires only about:

    $$log_2 \binom{1{,}000{,}000}{250{,}000} \approx 811{,}268 \text{ bits} \quad \rightarrow \quad \frac{811{,}268}{1{,}000{,}000} \approx 0.8113 \text{ bits/flip}$$

- **A biased sequence contains less "uncertainty" (Entropy) than an unbiased one, thus requiring fewer bits to transmit**

Efficient Encodings: The Coin Flip Case

- **Does Probability Affect Bit Count?**
  - **Fair Coin:** Heads and Tails occur with equal probability → **P(H) = P(T) = 1/2**
    - n flips → All 2^n sequences are equally likely
    - Therefore, identifying one sequence of n flips requires → $log_2 2^n = n$ bits
    - For 1,000,000 flips, requires 1,000,000 bits. → 1 bits/flip
  - **Biased Coin: P(H) = 1/4, P(T) = 3/4**
    - A typical sequence of 1,000,000 flips contains approximately:
      - 250,000 Heads, 750,000 Tails
    - The number of sequences with this composition is: $\binom{1{,}000{,}000}{250{,}000}$

    → So, Identifying one such sequence needs about:

    $log_2 \binom{1{,}000{,}000}{250{,}000} \approx 811{,}268$ bits → ≈ 0.8113 bits/flip
- **The more biased the probability distribution → the lower the Entropy → the fewer bits are required on average**

Efficient Encodings: The Coin Flip Case

Transmitting 1 Million Flips

- **Highly Biased Coin:** Suppose **P(H) = 0.001, P(T) = 0.999.**
  - Encode each flip separately (1 bit per flip): 1,000,000 flips → 1,000,000 bits
  - **Absolute Position Encoding**
    - describe mainly **where the rare H symbols occur**.
    - E.g., T T T T… T H T T T T T T T T T … T T T H
      ↑ ↑
    - Send only the positions of Heads (Heads: ~1,000 Tails: ~999,000).
    - Each position needs a 20-bit number. ($2^{20} \approx$ 1,000,000 positions)
    - 1000(Heads) x 20(bits) = 20000 bits → 20000/1,000,000 = 0.02bit/flip
  - Because Heads are very rare, this representation can be dramatically shorter.
    - The theoretical average information is only about 0.0114 bits/flip

Efficient Encodings: The Coin Flip Case

- **Why does this work**
  - Fair coin: H T H T T H …
    ↑ ↑ ← high uncertanty
  - Highly biased coin: T T T T … T H T T T T … T
    ↑↑↑↑ … ← highly predictable
  - If the next symbol is almost always T, observing another T gives us very little new information. (A common event → easy to predict)
  - A rare H is more surprising and therefore carries more info. (A rare event → surprising)
- **The amount of Information associated with an event should depend on probability**
  - A certain event, P(x)=1 → no information
  - Higher probability → less information
  - Lower probability → more information

Information Theory: Self-Information

- Information theory allows us to calculate the minimum bits required for *any* distribution
- **Self-Information:** $\mathrm{I}(x) = \log_2\left(\frac{1}{\mathrm{P}(x)}\right) = -\log_2(\mathrm{P}(x))$ (bits)
  - Self-information measures how surprising an event is.
  - It also corresponds to the ideal coding cost (number of bits) associated with that event.
  - Example

Event Probability	Self-Information
1	0 bits
1/2	1 bit
1/4	2 bits
1/1024	10 bits

High $\mathrm{P}(x)$ → small $\mathrm{I}(x)$

Low $\mathrm{P}(x)$ → large $\mathrm{I}(x)$

Information Theory: Entropy

- **Entropy** (Average Information)
  - Self-information tells us the information carried by one specific outcome. But a source can produce many different outcomes with different probabilities.
  - **Entropy** measures the average information per outcome produced by the source.

$$\boldsymbol{H}(x) = \sum P(x) \cdot I(x)$$

  - Ex. Biased Coin: P(H) = 1/4, P(T) = 3/4

→ Entropy $\boldsymbol{H}(x) = \frac{1}{4}\log_2(4) + \frac{3}{4}\log_2\left(\frac{4}{3}\right) \approx 0.8113$ bits/flip

- **The Entropy Limit**
  - Entropy represents the theoretical lower bound on the average number of bits needed to losslessly represent symbols from a source.

$$\bar{L} \geq H(x)$$

Information Theory: Sequence Coding

- **How can we use fractional bits?**
  - Fair 6-sided Die
    - Each face has probability P(x)=1/6
    - Self-info of each outcome is $\mathrm{I}(x) = \log_2(6)$ = 2.585 bits
    - But a single throw still requires 3 bits
  - How can we approach 2.585 bits/throw in practice?
  - Consider three throws together : 6^3=216 possible sequences
    - Since $2^7 < 216 < 2^8$

      → we can represent the three throws using 8 bits

      → 8/3 ≈ 2.67 bits/throw
  - By encoding longer sequence together, the average number of bits per throw can approach 2.585 bits.

⇨ Encode each separately
1 throw → 3 bits/throw

⇨ Encode 3 throws together
3 throws → 8 bits
→ 2.67 bits/throw

⇨ Encode long sequences
→ ~ 2.585 bits/throw

Fixed-Length vs. Variable-Length Coding

- **Ex. Biased 8-Symbol Source**
  - Consider a source with eight symbols:
  - P(A)= $\frac{1}{2}$, P(B)= $\frac{1}{4}$, P(C)= $\frac{1}{8}$, P(D)= $\frac{1}{16}$, P(E)= $\frac{1}{32}$, P(F)= $\frac{1}{64}$, P(G)= $\frac{1}{128}$, P(H)= $\frac{1}{128}$
- **Fixed-Length Coding**
  - With 8 possible symbols: $\log_2(8)$=3 → 3 bits for every symbol
  - A → 000, B→ 001, C→ 010, …
  - **Problem**: The probability distribution is not being exploited.
- **Variable-Length Coding**
  - Assign shorter codes to frequent symbols and longer codes to rare symbols:
  - **A :** 0, **B:** 10, **C:** 110, **D:** 1110, **E:** 11110, **F:** 111110, **G:** 1111110, **H:** 1111111
  - **Average code length**: $\bar{L} = \frac{1}{2}(1) + \frac{1}{4}(2) + \frac{1}{8}(3) + \frac{1}{16}(4) + \cdots =$ 1.984375 bits/symbol

saves ~34% of the bits

Fixed-Length vs. Variable-Length Coding

- **Ex. Biased 8-Symbol Source**
  - Consider a source with eight symbols:
  - P(A)= $\frac{1}{2}$, P(B)= $\frac{1}{4}$, P(C)= $\frac{1}{8}$, P(D)= $\frac{1}{16}$, P(E)= $\frac{1}{32}$, P(F)= $\frac{1}{64}$, P(G)= $\frac{1}{128}$, P(H)= $\frac{1}{128}$
- **Fixed-Length Coding**
  - With 8 possible symbols: $\log_2(8)$=3 → 3 bits for every symbol
  - A → 000, B→ 001, C→ 010, …
  - **Problem**: The probability distribution is not being exploited.
- **Variable-Length Coding**
  - Assume we assign variable-length code to each symbol according to its self-information: i.e., P(A)= $\frac{1}{2}$ → 1 bit, P(B)= $\frac{1}{4}$ → 2 bits, ….

→ **Average code length**: $\bar{L} = \frac{1}{2}(1) + \frac{1}{4}(2) + \frac{1}{8}(3) + \frac{1}{16}(4) + \cdots$ = 1.984375 bits/symbol

saves ~34% of the bits

Fixed-Length vs. Variable-Length Coding

- **How can we systematically construct efficient variable-length codes with minimum average code length?**
  - shorter codes → frequent symbols (high probability)
  - longer codes → rare symbols (low probability)
- **How can we ensure unambiguous decoding?**
  - Suppose we assign: A: 0, B: 01, C: 1
  - For the received bitstream: 01011
  - It can be decoded in different ways:
    - 0|1|0|1|1 → A,C,A,C,C
    - 01|01|1 → B, B, C
    - 01|0|1|1 → B, A, C, C

    Ambiguous decoding !

  → We need **prefix-free codes**:

  No codeword is allowed to be the prefix of another codeword.

Information Theory and Entropy Coding

- Information Theory
- Huffman Coding
- Arithmetic Coding
- Probability Model

Huffman Coding

- **Motivation**
  - In variable-length coding, frequent symbols should use shorter codewords and rare symbols can use longer codewords.
  - How can we systematically construct a prefix-free code with minimum average code length?
- **Huffman coding**
  - A particular type of optimal prefix-free code used for lossless data compression
  - A greedy algorithm that builds a bottom-up tree to determine the code for each symbol.
  - aims to minimize:

$$\bar{L} = \sum_i P(x_i)L(x_i)$$

$P(x_i)$: probability of symbol x_i

$L(x_i)$: length of its codeword

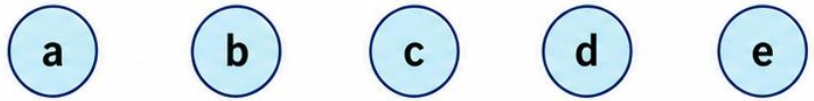
Core idea:
- Variable length coding (VLC)
- shorter codes → high-freq symbols
- longer codes → low-freq symbols

Huffman Coding – Algorithm

- **How to build the tree**
  - **Create leaf nodes**
    - Create one leaf node for each symbol.
    - Assign each node a weight equal to its frequency or probability.
  - **Combine the two smallest nodes**
    - Select the two nodes with the smallest weights and merge them into a parent node
    - Set the weight of the parent node to the sum of the two child weights
  - **Repeat**
    - Repeat the merging process until only one node left (and it becomes the root of the tree)
  - **Assign Bits**
    - Label the two branches of each internal node with 0 and 1 (e.g., left→0, right→1).
    - The sequence of bits along the path from the root to a leaf gives the symbol's codeword.

Huffman Coding – Example

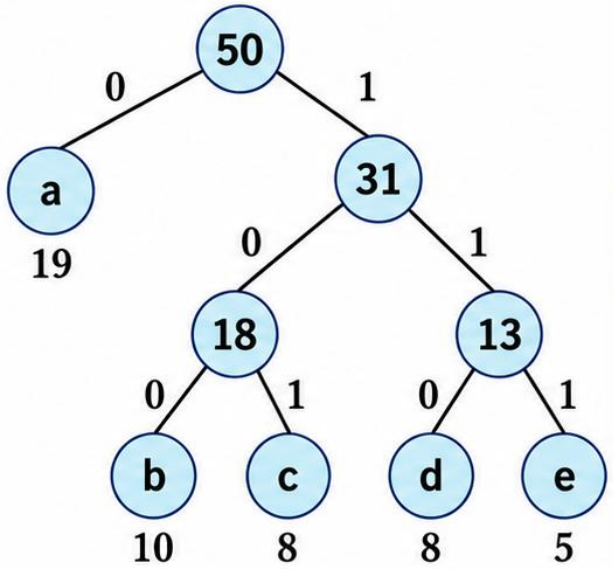
1. Show symbols as leaf nodes



Symbols	a	b	c	d	e
Frequency	19	10	8	8	5

2. Repeatedly combine the two lowest-frequency nodes

- Combine e(5) and d(8) → node 13
- Combine c(8) and b(10) → node 18
- Combine 13 and 18 → node 31
- Combine a(19) and 31 → root 50



More frequent symbols are placed closer to the root, so they use fewer bits.

Symbol	Code
a	0
b	100
c	101
d	110
e	111

Huffman Coding – Key Properties

- **Variable Length Coding**
  - The least probable symbols are merged first, so they tend to appear deeper in the Huffman tree → longer codewords.
  - More frequent symbols tend to appear closer to the root → shorter codewords
- **Prefix-Free Code**
  - No codeword is a prefix of any other codeword
    - E.g., if 0 is the codeword for symbol A, no other codeword can begin with 0.
  - Why is Huffman coding prefix-free ?
    - Every symbol is assigned to a **leaf node** of the binary tree.
    - No symbol is assigned to an internal node.
    - So, the path corresponding to one symbol cannot be the prefix of the path to another symbol.
  - This allows unique and immediate decoding without separators.

Symbol	Code
a	0
b	100
c	101
d	110
e	111

Huffman Coding

- **Advantage**
  - Huffman coding is optimal among binary prefix-free codes that assign one codeword to each source symbol. (symbol-by-symbol coding)
  - Easy to implement and can be decoded very quickly
- **Limitation**
  - **Integer Codeword Lengths**: Each codeword must be assigned an integer number of bits, while the ideal code lengths $-log_2(p_i)$ may be fractional.

    → Huffman coding does not always reach the entropy limit.

    When Does Huffman Reach the Entropy Limit?
    - If all symbol probabilities are powers of 1/2 ($p_i = 2^{-k}$, e.g. 1/2, 1/4, 1/8, …)

      → ideal code lengths : $-log_2(p_i)$ are integers.
    - In this case, Huffman coding reaches the entropy limit.
  - Codebook Overhead**:** The decoder must know the same probability model / Huffman tree as the encoder. Predefined codebook or transmitted in the stream.

Applications of Huffman Coding

- **Where is it used?**
  - **JPEG:** Huffman coding is commonly used in the final entropy-coding stage after transform and quantization.
  - **MP3:** Uses Huffman coding to compress quantized spectral coefficients..
  - **Deflate (ZIP/GZIP):** Combines LZ77 dictionary coding with Huffman coding.
  - **MPEG-1 /MPEG-2:** Use predefined variable-length coding tables for transform coefficients and other syntax elements.
  - **H.261 / H.263:** Also rely heavily on VLC tables for entropy coding.
  - **H.264/AVC:** Supports CAVLC (Context-Adaptive VLC) for transform coefficients, while CABAC provides higher coding efficiency.
  - **HEVC / VVC:** Mainly use CABAC rather than Huffman-style symbol-by-symbol VLC.

Information Theory and Entropy Coding

- Information Theory
- Huffman Coding
- Arithmetic Coding
- Probability Model

Arithmetic Coding

- **What is Arithmetic Coding?**
  - **Definition:** A form of variable-length entropy encoding that converts an entire message into a **single number** (a fraction) between 0 and 1.
  - Key Difference from Huffman:
    - Huffman: Assigns a fixed bit-string to each symbol (at least 1 bit)
    - Arithmetic: Encodes the entire sequence as a range, allowing for **fractional bits** per symbol.
  - **Optimality:** It can approach the **Entropy limit** more closely than Huffman, especially when symbol probabilities are very high (e.g., > 0.5).

Arithmetic Coding

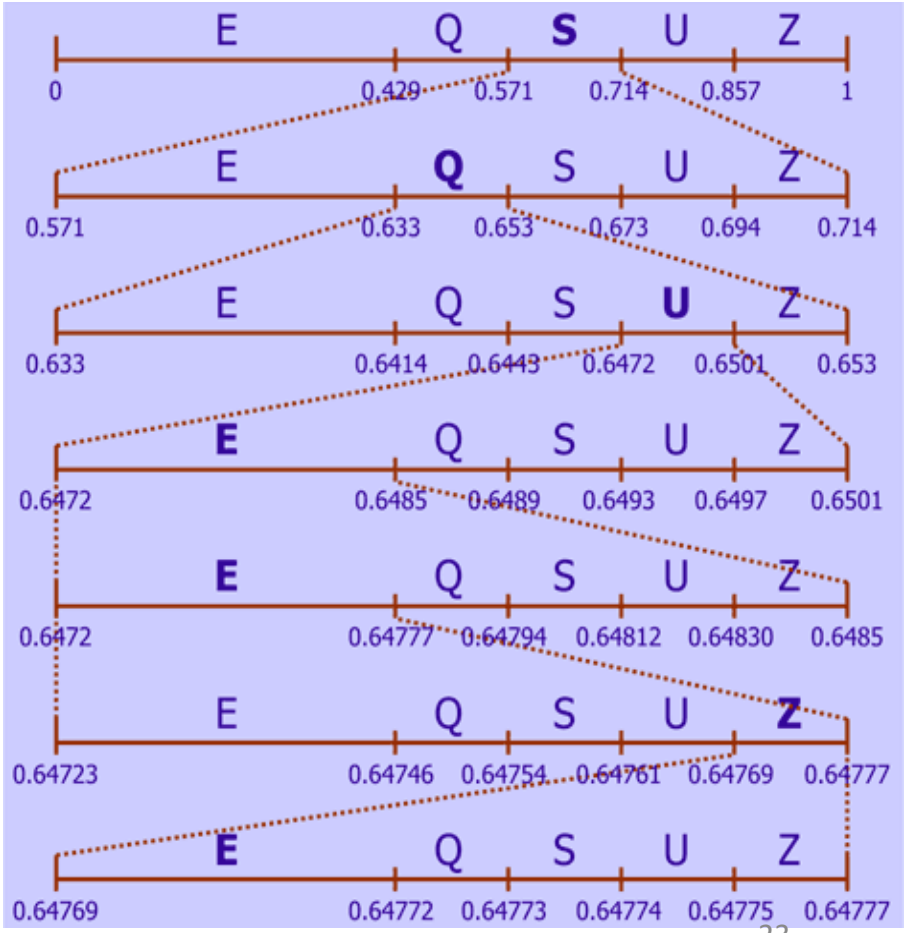
- **How it Works:**
  - **Initial Range:** Start with the interval [0, 1)
  - **Probability Mapping:** Each symbol is assigned a sub-interval based on its probability
  - **Recursive Subdivision**: Read a symbol → Narrow the current range to that symbol's sub-interval.
    - The next symbol's probabilities are then scaled into this new, smaller range.
  - **Final Output:** Any number (represented in binary) that falls within the final narrowed interval represents the entire message

Arithmetic Coding: "SQUEEZE"

- **Sequence to be encoded: SQUEEZE**
- Start with the interval [0, 1)
- Divide the interval into 5 subintervals, one for each symbol, with width proportional to their probabilities.
  - E: probability= 3/7 (≈0.429),
  - Each of others: probability = 1/7 (≈ 0.143)
- The first symbol, **S**, corresponds to [0.571~ 0.714)
- Next, subdivide current S interval using the same probability distribution.
- After encoding **Q**: → SQ is in [0.633~0.653)
- Repeat the subdivision process for remaining symbols.
- After encoding the entire string:
  - SQUEEZE is in [0.64769~0.64772)
- Any number inside this final interval can represent the sequence. The shortest binary fraction in this interval is :

  0.1010 01011101 (= 0.647705) → **only 12 bits**





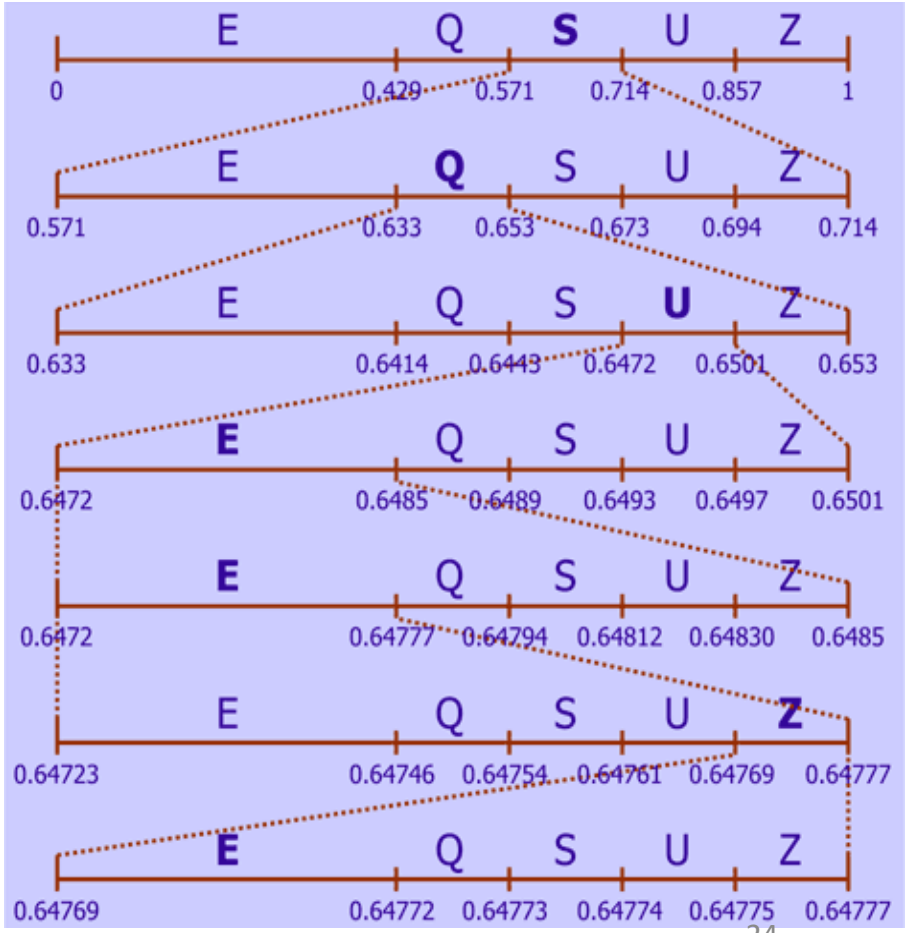
Arithmetic Decoding: "SQUEEZE"

- **The decoder receives the encoded value**

  $0.101001011101_2 = 0.647705078125$

- Start with the initial interval: [0, 1)
- divide it into the same symbol subintervals used by encoder.
- Since 0.647705 ∈ [0.571, 0.714), the first symbol is **S**.
- Next, subdivide current S interval using the same probability distribution.
- Since 0.647705 ∈ [0.633, 0.653), the second symbol is **Q**
- Repeat the same process: → SQUEEZE
- **When to stop**: either using sequence length or using a special end-of-sequence (EOS) symbol.
- **The decoder must use the same probability model as the encoder.**



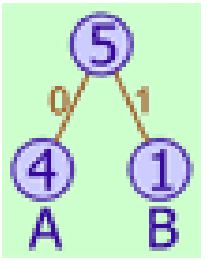


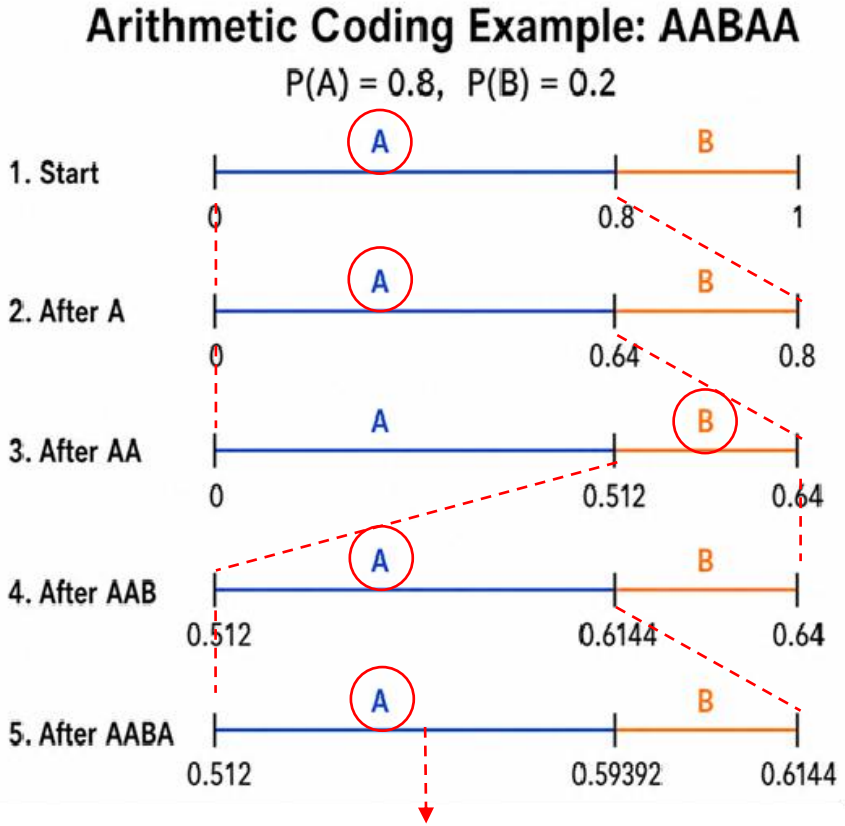
Huffman vs. Arithmetic coding

- **Example 2: Binary symbols ("AABAA")**
  - Symbol probabilities: P(A) = 0.8, P(B) = 0.2
- **Huffman coding**
  - Both symbols require **1 bit** → A: 1 bit , B: 1 bit
  - Total bit count: 1x4 + 1x1 = 5 bits
- **Arithmetic coding**
  - Encodes the **entire sequence as one interval**
  - AABAA can be represented by 0.1001 (0.5625). → 4 bits

- Although A occurs much more frequently than B, Huffman coding cannot assign less than one bit to A.
- Arithmetic coding can exploit the probability difference across the entire sequence.

Huffman coding





Arithmetic Coding

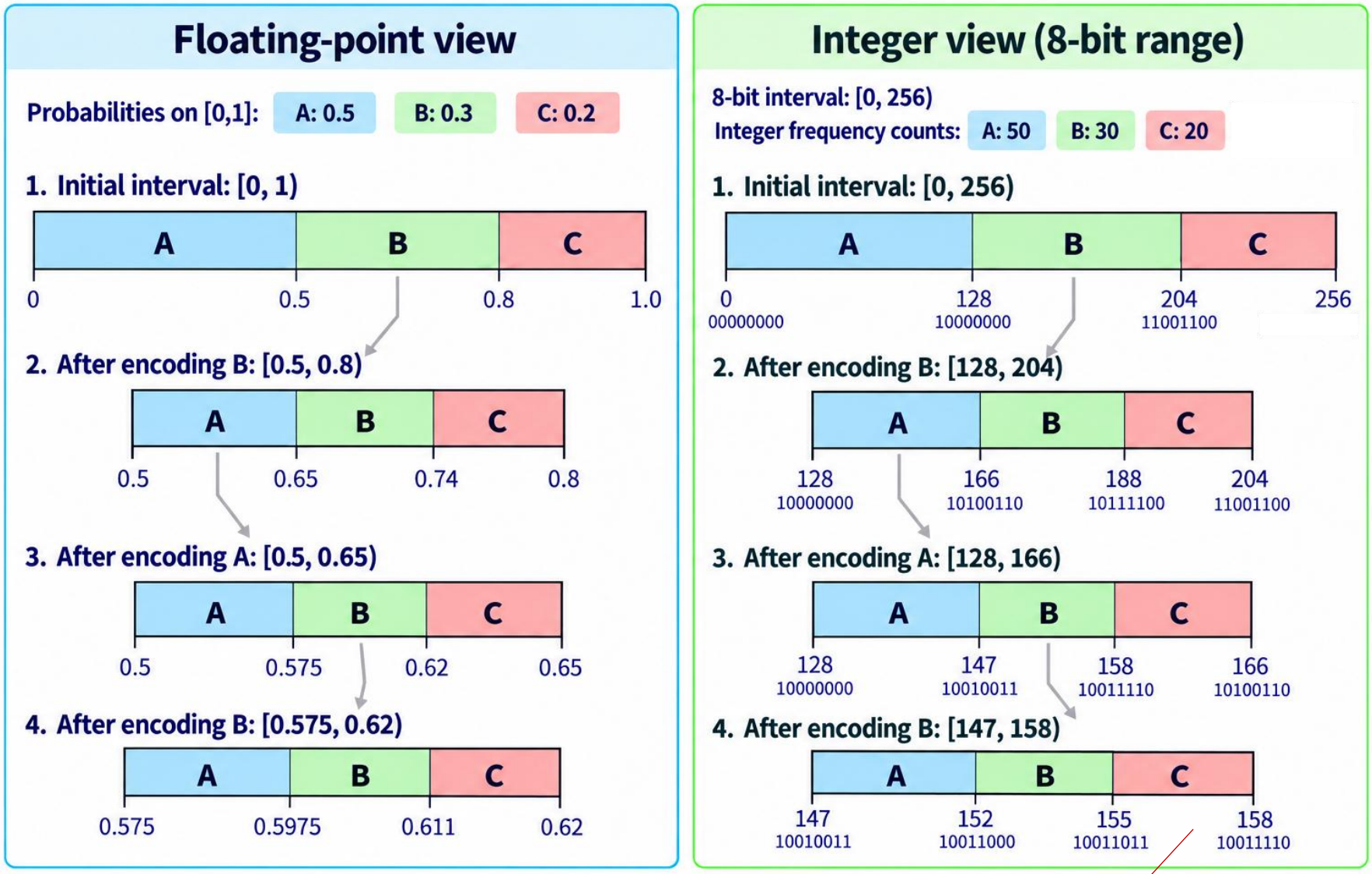
- **Why can arithmetic coding be more efficient?**
  - **Limitation of Huffman Coding**
    - Huffman coding assigns an integer number of bits to each symbol
    - If a symbol with a probability of 0.9, → $-\log_2(0.9) \approx 0.152$
    - But a Huffman codeword still requires 1 bit.
  - **Arithmetic Coding**
    - Arithmetic coding encodes the entire sequence jointly. it does not assign a separate bit string to each symbol.
    - Instead, each symbol progressively narrows the current interval.lity.
    - Highly probable symbols narrow the interval only slightly. → Therefore, their contribution to the final code length can effectively be less than one bit per symbol.
    - Particularly effective when symbol probabilities are highly skewed (some symbols have probabilities much larger than 0.5).

Arithmetic Coding: Integer Implementation

- **Challenges**
  - **Computational Complexity:** more expensive than Huffman's table look-up.
  - **Precision issue**
    - Theory: requires infinite precision (floating-point numbers)
    - Reality: digital hardware use finite-width registers
    - Problem: As the interval gets narrows, the difference between the "High" and "Low" boundaries becomes smaller than the smallest representable value, leading to **underflow**.
      - For example, the current interval : [0.10110100, 0.10110101)
  - **Solution: Integer implementation + Renormalization**
    - Instead of a range [0, 1), use unsigned integers
    - Expand the small interval back into a larger range to continue processing

Integer Implementation Example

Assume 8 bits are used.



What can we do if the interval become too narrow?

Arithmetic Coding: Integer Implementation

- **Renormalization**
  - **Check leading shared bits**: if most significant bits of L and H are identical, those bits are already fixed.
  - **Output the shared leading bit(s)** to stream
  - **Remove the shared prefix and left-shift the remaining bits**
    - If k leading bits are identical: output those k bits
    - L = (L << k) & 0xFFFF , & H = (H << k) & 0xFFFF (assume 16bits are used)
- Example
  - **Before**

    `L: 1011 0010 1010 0000`

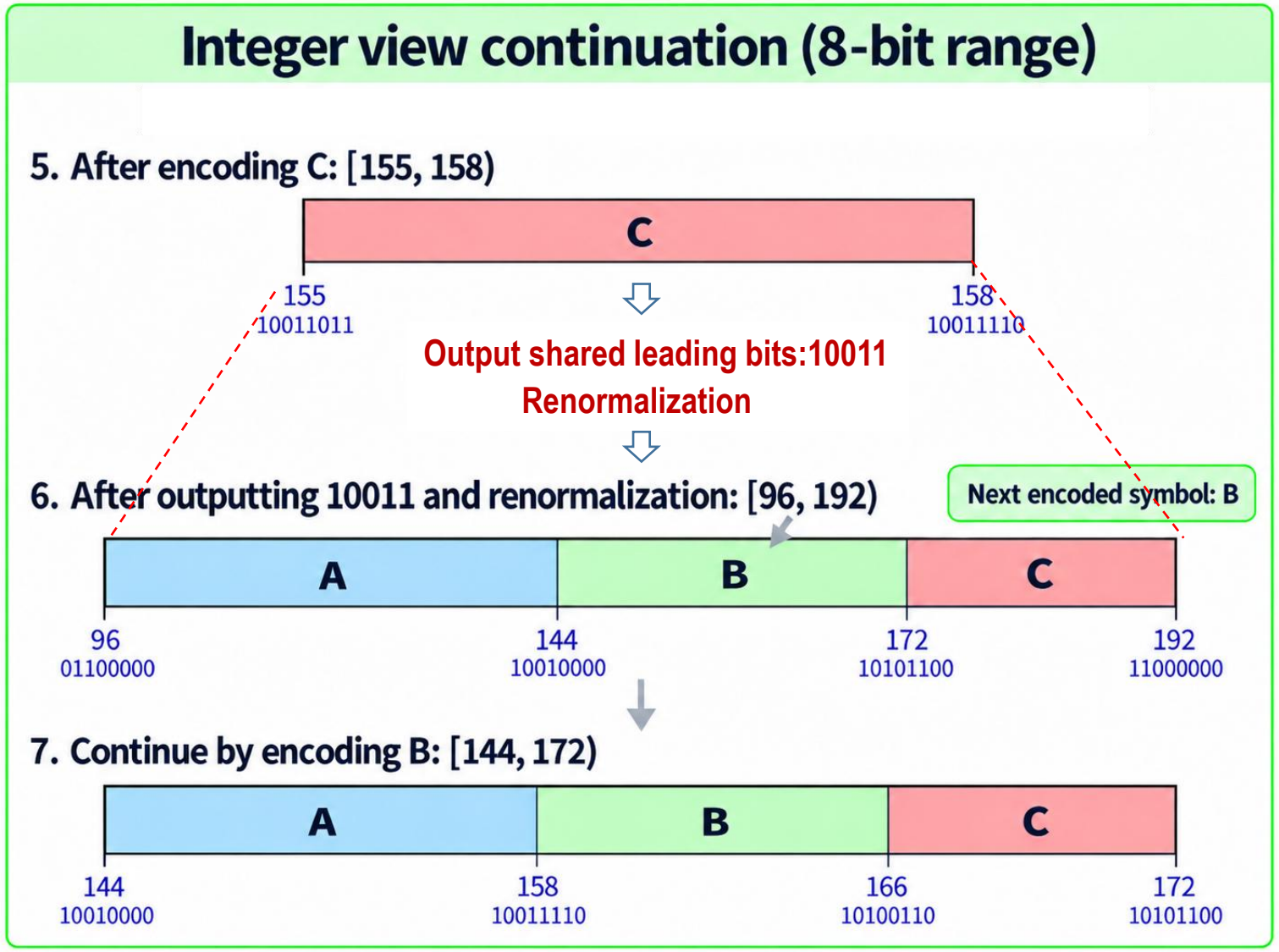
    `H: 1011 1101 0100 1111`

After

1. output **1011** to the bit-stream
2. → `L': 0010 1010 0000 0000`

   `H': 1101 0100 1111 0000`

Arithmetic Coding: Renormalization



Renormalization outputs the bits are already certain and expands the remaining interval to preserve precision.

Arithmetic Coding: Integer Implementation

- **Benefits of integer implementation**
  - Avoids floating-point rounding errors, ensuring that the encoder and decoder remain perfectly synchronized.
  - Only a few fixed-width integers/registers are needed to maintain the current interval and coding state.
  - Uses simple integer operations such as addition, subtraction, multiplication, comparison, and bit shifting, making it fast and hardware-friendly.
  - Through renormalization, bits that are already determined can be emitted immediately without waiting for the entire input sequence to be encoded.
    → incremental bit output

Information Theory and Entropy Coding

- Information Theory
- Huffman Coding
- Arithmetic Coding
- Probability Model

Probability Model

- **Why Probability Modeling?**
  - Entropy coders such as **Arithmetic Coding** require the probability of each symbol to determine the size of its coding interval.
  - The coding algorithm itself does **not** know these probabilities.
  - A **probability model** estimates the probability distribution of the symbols

$$P(x), \qquad \sum_{x} P(x) = 1$$

  - More probable symbols are assigned larger intervals, while less probable symbols receive smaller intervals.
  - Therefore, **better probability estimation leads to better compression efficiency**.

Probability Model

- **Static Probability Modeling**
  - A **static model** uses a fixed probability distribution throughout the entire encoding process. The distribution may be:
    - predefined based on prior knowledge, or
    - estimated from the entire input sequence before encoding.
  - The encoder and decoder must use exactly the **same probability model**.
  - If the probability distribution is estimated from the input, the model may need to be transmitted as **side information**.
    - Example: P(A) = 0.5, P(B) = 0.3, P(C) = 0.2
    - The arithmetic coder divides the current interval according to these probabilities.
  - Limitation: A fixed model cannot adapt when the statistical characteristics of the data change.

Probability Model

- **Adaptive Probability Modeling**
  - An **adaptive model** updates the probability estimates while symbols are being process. Probabilities can be estimated from previously observed symbol frequencies or weighted updates. For example:

$$P(x) = \frac{\text{count}(x)}{\sum_y \text{count}(y)}$$

  - Both the encoder and decoder perform the **same updates in the same order**.

    → Adaptive models can follow changes in the statistics of the input data.

    → the updated probability model does NOT need to be explicitly transmitted.
  - Advantages
    - No prior knowledge of the complete sequence is required.
    - Can adapt to changing data distributions.
    - Usually achieves better compression when the source statistics are non-stationary.

Context-Adaptive Probability Modeling

- A simple probability model estimates: P(x) using a single global distribution.
- However, symbols are often correlated with previously observed information.
- Instead of using a single global probability model, we can estimate: P(x|context).
- Different contexts use different probability models, where he context may include, for example:
    - neighboring coefficients,
    - coefficient position,
    - prediction mode,
    - block size,
    - previously coded symbols.

Assume binary symbol:
Context A: P(0) = 0.7, P(1) = 0.3
Context B: P(0) = 0.55, P(0) = 0.45
Context C: P(0) = 0.4, P(1) = 0.6
→ A, B, C are three probability models

- **Context → Probability model → Arithmetic Coding**
    - A better context makes the probability distribution more biased and predictable, allowing arithmetic coding to use fewer bits on average.

Context-Adaptive Probability Modeling

- **Why Does Context Improve Probability Modeling?**
  - Without context, the symbols may appear equally likely:
    - P(0) = 0.5, P(1) = 0.5
    - This distribution is not skewed and is highly uncertain.
    - The entropy is: H(X) = -0.5log_2(0.5) - -0.5log_2(0.5) = **1 bit/symbol**
  - Suppose the current symbol is strongly correlated with the previous symbol.
  - Using the previous symbol as context:
    - P(0| prev = 0) = 0.9, P(1| prev = 0) = 0.1
    - P(0| prev = 1) = 0.1, P(1| prev = 1) = 0.0
    - The entropy H(X|Context) = -0.9log_2(0.9) -0.1log_2(0.1) = **0.469 bit/symbol**

Context → More Predictable Distribution→ Low Entropy → Fewer bits